

Examples

Runnable, documented examples for the graphdb engine. **Everything below runs fully offline, CPU-only, with no model downloads** — each capability ships with a deterministic stub implementation behind the same trait as the real candle-transformers model, so you can run the demo today and swap in real weights later without changing your calling code.

Quick start

```
# from the workspace root
cargo run -p model-hub --example graph_backend
cargo run -p model-hub --example transcription
cargo run -p model-hub --example text_recognition
cargo run -p model-hub --example image_generation
cargo run -p model-hub --example web_scraping
cargo run -p model-hub --example image_segmentation
cargo run -p model-hub --example claude_code_agent
cargo run -p graphdb_rs --example jepa_world_model
```

Build them all at once:

```
cargo build -p model-hub --examples
cargo build -p graphdb_rs --example jepa_world_model
```

Index

Example	Crate	Capability	Trait	Stub	Real model (candle)
<code>graph_backend</code>	<code>model-hub</code>	Knowledge base: ingest, search, list, chunks	<code>GraphBackend</code>	<code>InMemory-Graph</code>	your graph engine
<code>jepa_world_model</code>	<code>graphdb_rs</code>	JEPA world model + latent/hybrid retrieval	—	<code>JEPAGraphRAG</code>	—
<code>text_recognition</code>	<code>model-hub</code>	OCR (text-line detection)	<code>TextRecognizer</code>	<code>StubTextRecognizer</code>	TrOCR
<code>image_generation</code>	<code>model-hub</code>	Text-to-image	<code>ImageGenerator</code>	<code>StubImageGenerator</code>	Stable Diffusion
<code>transcription</code>	<code>model-hub</code>	Speech-to-text	<code>AudioModel</code>	<code>StubAudioModel</code>	Whisper
<code>web_scraping</code>	<code>model-hub</code>	HTML → text/ links → KB	<code>WebScraper</code>	<code>pure-Rust</code>	+ <code>request</code> for live fetch
<code>image_segmentation</code>	<code>model-hub</code>	Per-pixel segmentation	<code>ImageSegmenter</code>	<code>StubImageSegmenter</code>	Segment Anything (SAM)
<code>claude_code_agent</code>	<code>model-hub</code>	Multi-agent coding assistant: difficulty router + skills + tool calling	<code>TextModel</code>	<code>StubTextModel</code>	quantized Llama (GGUF)

1. GraphBackend

The storage abstraction every pipeline talks to. Demonstrates ingesting documents (auto-chunked), semantic `search` (with an optional `doc_type` filter), `list_documents`, and `get_document_chunks`.

```
cargo run -p model-hub --example graph_backend
```

Swap the reference [`InMemoryGraph`] for the real engine by implementing the `GraphBackend` trait on your own store — the RAG and tool-calling pipelines are unchanged because they depend only on the trait.

2. JEPA (world model)

A **Joint-Embedding Predictive Architecture** for graphs, trained with the **VICReg** loss (invariance + variance + covariance) and combined with GraphRAG. The example builds a two-cluster graph, runs training steps (printing the VICReg terms), precomputes latent community embeddings, then does latent community search, latent node search, and a hybrid (local + global + latent) search.

```
cargo run -p graphdb_rs --example jepa_world_model
```

Note: the pure-Rust port computes the forward pass + VICReg loss and EMA-updates the target encoder, but does not run a gradient optimizer, so the reported loss is illustrative rather than a decreasing curve.

3. Text Recognition (OCR)

Detects horizontal text lines (bands of dark “ink” on a light background) and returns one `OcrLine` (text, confidence, bounding box) per line. The example renders a synthetic page and recovers three lines.

```
cargo run -p model-hub --example text_recognition
```

Production: implement `TextRecognizer` on top of **TrOCR** (`candle_transformers::models::trocr`) to turn each detected band into real text.

4. Image Generation

Turns a prompt + seed into a deterministic procedural image (a plasma field). Writes the result as a binary PPM (`generated.ppm`) — viewable directly or convertible with `convert generated.ppm generated.png` .

```
cargo run -p model-hub --example image_generation
```

Production: implement `ImageGenerator` on top of **Stable Diffusion** (`candle_transformers::models::stable_diffusion`).

5. Transcription (speech-to-text)

Synthesizes a 16 kHz mono tone and runs `transcribe` and `embed` through the `AudioModel` trait.

```
cargo run -p model-hub --example transcription
```

Production: load `WhisperModel::load(weights, config, tokenizer, mel_filters, device)` — the `transcribe / embed` calls are identical.

6. Web Scraping

Pure-Rust HTML parsing: extract the `<title>`, links (relative links resolved against a base URL), and clean body text; then ingest the readable text straight into an `InMemoryGraph` and query it.

```
cargo run -p model-hub --example web_scraping
```

Live fetching is intentionally excluded from the default build to keep the crate offline and lean. To fetch real URLs, add `request` in your binary and pass the HTML to `extract_text / ingest_into` (see `WebScraper::fetch` docs).

7. Image Segmentation

Partitions an image into `k` regions by quantizing per-pixel luminance into `k` bands. The example segments a gradient-plus-square image into 4 regions, prints each region's pixel area, and draws an ASCII preview of the mask.

```
cargo run -p model-hub --example image_segmentation
```

Production: implement `ImageSegmenter` on top of **Segment Anything (SAM)** (`candle_transformers::models::segment_anything`) for prompt-driven instance masks.

8. Claude-Code agent (multi-agent + skills + tool calling)

A coding assistant in the spirit of **Claude Code**, grounded in this codebase. It brings together three ideas:

1. **A main orchestrator that delegates by difficulty.** Each task is classified into `Simple / Moderate / Complex` and routed across two engines — the deterministic **Needle** engine for cheap work and a **local LLM** for hard reasoning:

Difficulty	Route tool with	Compose answer with	Skill injected
Simple	Needle	deterministic (no LLM)	no
Moderate	Needle	local LLM	yes
Complex	local LLM	local LLM	yes

1. **Tool calling.** An explicit, visible tool-use loop over the `graphdb_tool_schemas registry ( search_knowledge_base , list_documents , get_document_chunks )`. Each turn prints the `tool_use` call and a preview of the `tool_result`, then composes the answer from it.
2. **Skills.** Reusable instruction “folders” on disk under `examples/skills/<name>/SKILL.md`, each with a description, trigger `cues`, and an instruction body. The best-matching skill for a task is injected into the LLM prompt — the same pattern as Claude Code’s Agent Skills. Add a skill by dropping in a new `SKILL.md`; no code changes.

```
# fully offline (deterministic Needle + stub LLM):
cargo run -p model-hub --example claud_code_agent

# with a real local model (quantized Llama GGUF):
GRAPHDB_MODEL=/path/to/model.gguf \
GRAPHDB_TOKENIZER=/path/to/tokenizer.json \
cargo run -p model-hub --example claud_code_agent
```

With the default offline stub the LLM tiers emit placeholder prose (tool results are still real); plug in a GGUF via the two env vars for real answers on the Moderate/Complex tiers.

GPU / real models

All GPU backends are **off by default**; the crate builds and runs CPU-only out of the box. Enable a backend with a cargo feature when you load real weights:

```
cargo run -p model-hub --example transcription --features cuda # NVIDIA
cargo run -p model-hub --example transcription --features metal # Apple
```

See each example’s module-level docs (`//!`) for the exact real-model loading snippet.